

FraudWatch

Product Requirements Document

Author	Shrijani (Diya) Manna · Duke MEM '26
GitHub	github.com/diyaboop/fraud-watch
Version	1.0
Status	Draft
Date	June 14, 2026
Audience	Product managers, ML engineers, compliance teams, hiring reviewers

1 · Overview

FraudWatch is an end-to-end fraud detection, bias audit, and model monitoring platform built on the IEEE-CIS Kaggle dataset (590,540 financial transactions, 3.5% fraud rate). It trains a Random Forest classifier, surfaces demographic disparities in false positive rates across card type, product category, and billing region, and tracks weekly performance drift — all served through a FastAPI backend with a live dashboard.

The system is designed to demonstrate responsible AI development: it does not merely optimize for model accuracy but surfaces where automated decisions may systematically disadvantage certain transaction profiles, enabling human oversight and remediation.

2 · Problem Statement

Background

Fraud detection systems flag millions of transactions daily. False positives — legitimate transactions wrongly blocked — impose real customer harm: declined payments, frozen accounts, and loss of trust. When these errors are unevenly distributed across customer segments, the system creates a de-facto disparate impact even without any intent to discriminate.

Current State

Most deployed fraud models are evaluated purely on aggregate metrics (AUC, precision, recall). Demographic subgroup analysis is rarely part of the standard model card or release process. There is no live bias monitoring in production for most mid-market financial institutions.

Opportunity

FraudWatch demonstrates that bias auditing can be a first-class engineering concern — automated, reproducible, and queryable via API. The 45,375 false positives surfaced in a single test window, with a 65% disparity between credit and debit cardholders, represent a concrete, measurable harm that this system makes visible and actionable.

3 · Goals & Success Metrics

Metric	Baseline	Target	Status
Fraud recall (test set)	—	≥ 0.85	✓ 0.92
FPR disparity surfaced (card type)	—	Quantified + significant	✓ Z=66.6, p<0.001
FPR disparity surfaced (product)	—	Quantified + significant	✓ Z=109.1, p<0.001
Weekly drift detection	—	0 undetected drift weeks	✓ 0 / 12
API response time (p95)	—	< 500ms	In scope
Audit reproducibility	—	Deterministic from seed	✓ random_state=42
Dashboard load time	—	< 3s on local network	In scope

4 · Users & User Stories

ML Engineer / Data Scientist

Trains and evaluates the fraud model. Needs reproducible pipelines, interpretable bias metrics, and drift alerts before production incidents.

Compliance / Risk Officer

Responsible for regulatory adherence (ECOA, CFPB, SR 11-7). Needs documented evidence of bias monitoring and remediation workflows.

Product Manager / Analyst

Owns the fraud product roadmap. Needs a clear dashboard summarizing model health, false positive burden, and actionable audit findings.

Epic 1 — Model Training & Evaluation

- As an **ML engineer**, I want a single-command pipeline that trains, evaluates, and saves the model so that onboarding new engineers takes under an hour.
- As an **ML engineer**, I want class-balanced training so that the 3.5% fraud rate does not cause the model to trivially predict 'not fraud' for everything.
- As a **PM**, I want a confusion matrix and classification report in the logs so that I can understand precision/recall trade-offs without writing code.

Epic 2 — Bias Audit

- As a **compliance officer**, I want false positive rates computed per card type so that I can assess ECOA disparate-impact risk before deployment.
- As a **compliance officer**, I want statistically significant disparity tests (Z-test, p-values) so that findings are defensible in a regulatory review.

- As an **ML engineer**, I want audit results persisted to SQLite so that I can query historical disparity trends without re-running the full pipeline.

Epic 3 — Monitoring & API

- As a **PM**, I want a live dashboard showing weekly recall and FPR so that I can brief leadership on model health without a data science handoff.
- As an **ML engineer**, I want drift alerts flagged automatically so that degradation is caught before it reaches end-users.
- As a **compliance officer**, I want a /predict endpoint so that individual transaction decisions can be audited in real time.

5 · Functional Requirements

ID	Priority	Requirement
FR-01	Must	Train Random Forest on merged transaction + identity data
FR-02	Must	Apply class-balanced weights to handle 3.5% fraud minority class
FR-03	Must	Save model artifact and feature list to outputs/ via joblib
FR-04	Must	Compute FPR, FNR, and recall per card4, card6, ProductCD, addr2
FR-05	Must	Run proportions Z-test between key segment pairs
FR-06	Must	Persist audit_results and bias_summary to SQLite
FR-07	Must	Compute weekly precision, recall, FPR across 12 simulated weeks
FR-08	Must	Flag drift when recall < 0.85 or FPR > 0.55
FR-09	Must	Expose /metrics, /audit, /monitor, /predict via FastAPI
FR-10	Must	Serve single-page dashboard at GET /
FR-11	Should	Return bias charts as base64 PNG embedded in JSON
FR-12	Should	Containerize full app in a single Dockerfile
FR-13	Could	Accept custom threshold parameter in /predict
FR-14	Won't	Real-time streaming from live transaction feeds (v2 scope)

6 · Non-Functional Requirements

Performance: API p95 latency < 500ms; dashboard first paint < 3s on local network.

Reproducibility: All training, audit, and monitor runs produce identical outputs with random_state=42.

Portability: Docker image runs on any Linux/macOS host with Docker installed; no GPU required.

Security: No PII in API responses; model artifact not exposed directly via HTTP.

Observability: Drift alerts surfaced in /monitor JSON response and in uvicorn server logs.

Maintainability: ML pipeline (train.py, audit.py, monitor.py) untouched by API layer; clean separation of concerns.

7 · System Design

Architecture

Layer	Component	Technology
Data	IEEE-CIS CSVs (590K rows)	Pandas, CSV
Training	Random Forest (100 trees, depth=10)	Scikit-learn
Audit	FPR disparity + Z-tests	SciPy / statsmodels
Monitoring	12-week drift simulation	Pandas, qcut
Persistence	Model + metrics + audit results	jobjlib, SQLite
API	REST endpoints (/metrics /audit ...)	FastAPI, Uvicorn
Frontend	Single-page dashboard	Vanilla JS, HTML/CSS
Deployment	Containerized app	Docker

API Endpoints

Method	Path	Description	Auth
GET	/metrics	Aggregate KPIs (recall, precision, FPR, totals)	None
GET	/audit	Bias audit data + base64 chart PNG	None
GET	/monitor	Weekly metrics + drift flags + chart	None
POST	/predict	Score single transaction (30 features)	None
GET	/	Serve SPA dashboard (index.html)	None

Data Flow

1. train.py reads CSVs → feature engineering → model fit → saves fraud_model.pkl + features.pkl
2. audit.py loads model → runs inference on test split → computes per-segment FPR → writes to fraudwatch.db
3. monitor.py loads model → simulates 12 weekly batches → computes drift metrics → writes weekly_metrics to fraudwatch.db
4. api.py reads fraudwatch.db at request time → generates charts on-the-fly → returns JSON

8 · Acceptance Criteria

AC-01 · Training pipeline

Given the IEEE-CIS CSV files exist in data/
When train.py is executed,
Then outputs/fraud_model.pkl and outputs/features.pkl are created, model recall on the test set exceeds 0.85, and the run completes without error.

AC-02 · Bias audit persistence

Given fraud_model.pkl exists,
When audit.py is executed,
Then fraudwatch.db contains tables audit_results and bias_summary with non-empty rows, and the credit vs debit Z-stat is ≥ 60 .

AC-03 · Drift monitoring

Given fraud_model.pkl exists,
When monitor.py is executed,
Then fraudwatch.db contains 12 rows in weekly_metrics and drift_flag is 0 for all weeks given stable simulated data.

AC-04 · API liveness

Given the FastAPI server is running on port 8000,
When GET /metrics is called,
Then the response status is 200 and contains avg_recall, avg_fpr, total_transactions.

AC-05 · Predict endpoint

Given the server is running,
When POST /predict is called with a valid 30-feature JSON payload,
Then the response includes fraud_probability (0–1), flagged (bool), and risk_level (LOW/MEDIUM/HIGH).

AC-06 · Docker build

Given Docker is installed,
When 'docker build -t fraudwatch .' is run,
Then the image builds without error and 'docker run -p 8000:8000 fraudwatch' serves all endpoints.

9 · Risks & Open Questions

Risk	Level	Mitigation
Simulated weeks from a single test split — not true temporal drift	Medium	Label as 'simulated'; v2 to use rolling time windows from TransactionDT
High FPR (39.8% avg) due to 0.30 probability threshold	High	Threshold is tunable; surfaced explicitly in dashboard and docs

Risk	Level	Mitigation
card6 and ProductCD encoding (int codes) may not generalize	Medium	Encodings are deterministic from same data split; document in model card
No authentication on API endpoints	Low	Localhost / internal use only; add API key middleware for production
Large CSV data files not included in Docker image	Medium	Pre-computed outputs/ shipped; retrain requires data re-mount
Non-US FPR finding (77–85%) based on addr2 codes — Medium	Medium	Document as proxy signal; pair with demographic validation in production

Open Questions

- Should the 0.30 fraud probability threshold be configurable per product line?
- What is the right FPR disparity threshold to trigger a mandatory model review?
- How should the system handle missing identity data (left-join nulls → -999 fill)?
- Is addr2 a reliable proxy for billing region, or should IP geolocation be used?

10 · Out of Scope & Regulatory Mapping

Out of Scope (v1)

- Real-time transaction streaming or Kafka integration
- Model retraining automation (MLflow, Airflow)
- Multi-model A/B testing or shadow deployment
- End-user authentication and role-based access control
- Production-grade alerting (PagerDuty, Slack webhooks)

Regulatory Mapping

Regulation	Applicability	How FraudWatch Addresses It
ECOA (Equal Credit Opportunity Act)	High	FPR disparity audit across card type surfaces potential disparate impact in credit-adjacent decisions
EU AI Act (High-Risk AI, Annex II)	High	Bias audit, documentation, and human oversight mechanisms directly map to Art. 9–13 requirements
CFPB Unfair, Deceptive, or Abusive Practices	Medium	False positive burden on customers constitutes potential harm; quantification enables remediation
SR 11-7 (Model Risk Management)	High	Weekly performance monitoring, drift detection, and documented acceptance criteria fulfill SR 11-7